



Here are essential JavaScript and React interview questions, broken down into core concepts from foundational to advanced.

● Core JavaScript Questions

1. What is the Event Loop in JavaScript, and how does asynchronous execution work?

- **Answer:** JavaScript is single-threaded, meaning it executes one command at a time on the **Call Stack**. Asynchronous operations (like `setTimeout`, `fetch`, or event listeners) are delegated to the browser's Web APIs.
- When complete, their callbacks enter either the **Microtask Queue** (Promises, `queueMicrotask`) or the **Macrotask Queue** (`setTimeout`, `setInterval`). The Event Loop constantly checks if the Call Stack is empty; if so, it pushes all microtasks first before processing the next macrotask.

2. What is the difference between `var`, `let`, and `const`?

- `var`: Function-scoped, hoisted to the top of its scope with `undefined`, and can be re-declared.
- `let`: Block-scoped, hoisted to the top of its block but placed in a **Temporal Dead Zone (TDZ)** until declared; cannot be re-declared in the same scope.
- `const`: Block-scoped like `let`, cannot be reassigned or re-declared, and must be initialized upon declaration.

3. Explain Closures in JavaScript with a practical example.

- **Answer:** A **closure** is a function that retains access to variables in its outer lexical scope even after the outer function has finished executing.

JavaScript



```
function createCounter() {  
  let count = 0;  
  return function() {  
    count++;  
    return count;  
  };  
}  
  
const counter = createCounter();  
console.log(counter()); // 1  
console.log(counter()); // 2
```

● Core React Questions

1. How does the Virtual DOM work in React, and what is Reconciliation?

- **Answer:** The Virtual DOM (VDOM) is a lightweight in-memory representation of the real DOM. Wordsmith Creations
- When state or props change, React generates a new VDOM tree. It then performs a diffing algorithm (**Reconciliation**) to compare the new VDOM with the previous one, computing the minimum number of changes required and batch-updating only those parts in the real DOM. GreatFrontEnd

2. What is the difference between State and Props?

- **Props:** Read-only data passed from a parent component down to a child component. Wordsmith Creations
- **State:** Local, mutable data managed internally inside a component that can change over time in response to user events or network requests. Wordsmith Creations

3. What are the main React Hooks and their common use cases?

Hook	Primary Purpose
<code>useState</code>	Manages local component state.
<code>useEffect</code>	Handles side effects (data fetching, DOM listeners, subscriptions).
<code>useContext</code>	Consumes values from React Context to avoid prop drilling.
<code>useRef</code>	Persists values across renders without causing re-renders or accesses DOM nodes directly.
<code>useMemo</code> / <code>useCallback</code>	Memoizes computed expensive values or function instances to optimize rendering performance.

⚡ Senior & Performance Questions

1. What causes unnecessary re-renders in React, and how do you prevent them?

- **Causes:** Parent re-renders, unstable object/array prop references, or inline functions being recreated on every render.
- **Solutions:**
 1. Wrap components in `React.memo`. foundit
 2. Memoize callback props using `useCallback`. foundit
 3. Memoize heavy computations using `useMemo`. foundit
 4. Keep state co-located as close as possible to where it is needed. Wordsmith Creations

2. What is "Prop Drilling," and how do you avoid it?

- **Answer:** Prop drilling occurs when data is passed down through multiple layers of intermediate components that don't need the data themselves. Wordsmith Creations
- **Solutions:** Use the **React Context API**, component composition (passing children/slots), or state management libraries (such as Zustand or Redux). Wordsmith Creations

Are you preparing for a specific role level (e.g., Junior, Mid, Senior) or focusing on a particular area like system design, coding challenges, or modern features like React 19 / Next.js?

Top React Interview Questions & Answers

This video covers high-frequency React interview questions and concepts asked by companies for candidates across different experience levels.

give me 2026 react and javascript interviews questions give me about 50 questions in pdf

While I cannot directly output or stream an attachable `.pdf` binary file, you can **save this structured guide directly as a PDF** right from your browser:

How to save as PDF:

Press `Ctrl + P` (or `Cmd + P` on Mac) → Select **"Save as PDF"** under Destination → Click **Save**.

Top 50 JavaScript & React Interview Questions & Answers

Part 1: JavaScript Questions (Q1 – Q25)

Core Fundamentals & ES6+

Q1. What is the Temporal Dead Zone (TDZ) in JavaScript?

- **Answer:** TDZ is the period between entering a scope and the point where a `let` or `const` variable is declared and initialized. Accessing a variable in its TDZ throws a `ReferenceError`.

Q2. Explain Closures with a practical use-case.

- **Answer:** A closure is a function that retains access to variables from its outer lexical scope even after the outer function has finished executing.

JavaScript

```
function createCounter() {  
  let count = 0;  
  return () => ++count;  
}  
const counter = createCounter();  
console.log(counter()); // 1  
console.log(counter()); // 2
```

Q3. What is Hoisting, and how do functions vs. `var` / `let` / `const` differ?

- **Answer:** Hoisting moves declarations to the top of their scope prior to execution. Function declarations are hoisted completely with implementation. `var` is hoisted and initialized to `undefined`. `let` and `const` are hoisted but remain uninitialized in the TDZ.

Q4. What is the difference between `==` and `===` ?

- **Answer:** `==` (abstract equality) performs implicit type coercion before comparison. `===` (strict equality) compares both value and data type without coercing types.

Q5. Explain Event Delegation and why it is useful.

- **Answer:** Event delegation relies on event bubbling to attach a single event listener to a parent element rather than attaching listeners to every individual child node. This conserves memory and handles dynamic elements seamlessly.

Q6. What is the difference between shallow copy and deep copy?

- **Answer:** A **shallow copy** duplicates top-level properties; nested objects/arrays still reference the same memory locations (e.g., `{ ...obj }` or `Object.assign()`). A **deep copy** recursively duplicates all nested levels (e.g., `structuredClone(obj)` or `JSON.parse(JSON.stringify(obj))`).

Q7. What are `call`, `apply`, and `bind`?

- `call()`: Invokes a function immediately, passing `this` explicitly and individual arguments (`fn.call(ctx, arg1, arg2)`).
- `apply()`: Invokes a function immediately, passing `this` and arguments as an array (`fn.apply(ctx, [arg1, arg2])`).
- `bind()`: Returns a new function with `this` permanently bound without executing it immediately.

Q8. What is the behavior of `this` in regular functions vs. arrow functions?

- **Answer:** In regular functions, `this` is dynamically bound based on *how* the function is invoked. Arrow functions do not possess their own `this`; they capture `this` lexically from their enclosing scope.

Q9. Explain Prototype Chaining in JavaScript.

- **Answer:** Every JavaScript object has an internal `[[Prototype]]` link to another object. When accessing a property on an object, JS checks the object itself, then traverses up its prototype chain until found or reaching `null`.

Q10. What is a WeakMap/WeakSet, and when would you use it?

- **Answer:** `WeakMap` and `WeakSet` hold weak references to object keys/values. If an object referenced inside a `WeakMap` has no other references, it is eligible for garbage collection, preventing memory leaks in caching or metadata tracking.

Asynchronous JavaScript & Event Loop

Q11. Explain the JavaScript Event Loop, Call Stack, Microtask, and Macrotask Queues.

- **Answer:** Single-threaded JS executes synchronous code via the **Call Stack**. Asynchronous tasks delegate work to Web APIs. When ready:
 1. Promises and `queueMicrotask` enter the **Microtask Queue**.
 2. `setTimeout`, `setInterval`, and I/O enter the **Macrotask Queue**.The Event Loop empties the Call Stack, flushes *all* Microtasks, then processes *one* Macrotask.

Q12. What is the output of this code?

JavaScript



```
console.log('1');  
setTimeout(() => console.log('2'), 0);
```

```
Promise.resolve().then(() => console.log('3'));
console.log('4');
```

- **Answer:** Output: 1, 4, 3, 2. Microtask (Promise) executes before Macrotask (setTimeout).

Q13. How does async/await work under the hood?

- **Answer:** async/await is syntactic sugar built over Promises and Generator functions. await pauses function execution inside an asynchronous context, returning control to the Event Loop until the Promise settles.

Q14. What is Promise.allSettled() vs. Promise.all() ?

- **Answer:** Promise.all() fails fast and rejects as soon as any single promise rejects. Promise.allSettled() waits for all promises to settle (fulfill or reject) and returns an array describing the outcome of each.

Q15. How do you cancel an active fetch request in modern JavaScript?

- **Answer:** Using the AbortController API:

JavaScript

```
const controller = new AbortController();
fetch('/api', { signal: controller.signal });
controller.abort(); // Cancels request
```

Performance & Modern ECMAScript (ES2024–ES2026)

Q16. What is Debouncing vs. Throttling?

- **Debouncing:** Delays function execution until a specified delay has elapsed since the last time the event was fired (e.g., search bar inputs).
- **Throttling:** Ensures a function is executed at most once in a specified time interval (e.g., scroll/resize listeners).

Q17. What are ES2025 Iterator Helpers?

- **Answer:** Modern JavaScript adds chainable methods directly to iterables (map , filter , take , drop , toArray), enabling lazy-evaluation data processing without needing utility libraries like Lodash. igmGuru

Q18. What are Built-in Set operations in modern JS?

- **Answer:** Native methods added to Set objects for set mathematics, such as setA.union(setB) , setA.intersection(setB) , and setA.difference(setB) .

Q19. What is Promise.try() ?

- **Answer:** Promise.try(fn) executes a function synchronously and wraps its return value or thrown error into a Promise, handling both synchronous exceptions and async errors cleanly in one chain. igmGuru

Q20. Explain Web Workers and when to use them.

- **Answer:** Web Workers allow running JS scripts in background threads independent of the main UI thread. They prevent main-thread freezing during heavy CPU computation.

Q21. How do ES Modules (`import / export`) work compared to CommonJS (`require`)?

- **Answer:** ES Modules are static, evaluated at compile/parse-time, and support tree-shaking. CommonJS modules are loaded dynamically at runtime synchronously.

Q22. What is `Object.freeze()` vs `Object.seal()` ?

- **Answer:** `Object.freeze()` prevents adding, deleting, or modifying existing properties. `Object.seal()` allows modifying existing property values but prevents adding or deleting properties.

Q23. What is a Memory Leak in JavaScript, and how do you prevent it?

- **Answer:** Unwanted retained memory caused by uncleared timers (`setInterval`), global variables, retained DOM references, or unremoved event listeners.

Q24. How do Generator Functions (`function*` and `yield`) work?

- **Answer:** Generators can pause (`yield`) and resume execution. Calling a generator returns an iterator object with a `.next()` method.

Q25. What is `structuredClone()` and why prefer it over `JSON.parse(JSON.stringify())` ?

- **Answer:** `structuredClone()` is a native JS deep-cloning mechanism that supports cyclic references, `Map` , `Set` , `Date` , `ArrayBuffer` , and TypedArrays, which JSON serialization breaks or ignores.

Part 2: React Questions (Q26 – Q50)

Fundamentals & Core Rendering Engine

Q26. How does the Virtual DOM and Reconciliation algorithm work in React?

- **Answer:** React keeps a light representation of the DOM tree in memory. When state updates, React creates a new Virtual DOM, compares it with the previous snapshot using its diffing algorithm (**Fiber**), and updates only modified real DOM nodes.

Q27. What is React Fiber?

- **Answer:** React Fiber is the rendering engine rewrite introduced to break reconciliation work into incremental units. It allows React to pause, prioritize, resume, or discard render updates to maintain fluid UI responsiveness.

Q28. Why must `key` props be unique in dynamic lists?

- **Answer:** `key` identifiers tell React which items were changed, added, or removed. Using unstable indices can cause state bugs and unnecessary re-renders when list ordering changes.

Q29. Controlled vs. Uncontrolled Components?

- **Controlled:** Form input values are managed by React state (`value={state}` + `onChange`).
- **Uncontrolled:** Form values are handled natively by the DOM and accessed via `useRef` .

Q30. What is automatic batching in modern React?

- **Answer:** React groups multiple state updates triggered inside event handlers, promises, or timeouts into a single re-render for optimal UI performance.

Q31. What are the core Rules of Hooks?

○ **Answer:**

1. Call hooks **only at the top level** of React functional components (not inside loops or conditions).
2. Call hooks **only from React function components or custom hooks**.

Q32. `useEffect` vs `useLayoutEffect` ?

- `useEffect` : Asynchronous/non-blocking; runs *after* browser paint.
- `useLayoutEffect` : Synchronous/blocking; fires *before* the browser paints to measure DOM layouts and prevent layout flickering.

Q33. `useMemo` vs `useCallback` ?

- `useMemo` : Caches and returns the **result of a calculation**.
- `useCallback` : Caches a **function instance** across re-renders.

Q34. When should you use `useReducer` over `useState` ?

- **Answer:** `useReducer` is preferred for complex state logic involving multiple sub-values, when the next state depends on the previous state, or when state transitions follow Redux-like patterns.

Q35. What is `useRef` and when does updating it cause a re-render?

- **Answer:** `useRef` returns a mutable object with a `.current` property that persists across renders. Updating `.current` **never triggers a component re-render**.

Wordsmith Creations

Q36. What is `useId` and why is it useful?

- **Answer:** A hook for generating unique IDs that remain stable across server and client rendering, eliminating SSR hydration mismatch errors for form field labels and ARIA attributes.

Q37. `useTransition` vs `useDeferredValue` ?

- `useTransition` : Wraps a state setter function to mark state updates as non-urgent transitions (keeping input reactive while background processing happens).

Wordsmith Creations

- `useDeferredValue` : Wraps a dynamic value directly to defer updating derived non-critical components.

Architecture & Modern Features (React 19+)

Q38. What is the React 19 Compiler (React Forget)?

- **Answer:** A build-time compiler that automatically handles component memoization, effectively eliminating the manual need for `useMemo` and `useCallback` optimizations in component code. Wordsmith Creations

Q39. How are `ref` props simplified in React 19?

- **Answer:** `ref` can be passed directly as a standard prop to functional components without needing `forwardRef()`.

Q40. What are React Server Components (RSC)?

- **Answer:** Components that execute and render purely on the server. They emit no JS bundle to the client, reducing download size and keeping data-fetching logic server-side. [Wordsmith Creations](#)

Q41. What is the `use()` API introduced in React 19?

- **Answer:** A hook-like function that reads resources (promises or context) conditionally inside render functions, unlocking flexible async handling unlike standard hooks. [GitHub](#)

Q42. What are React 19 Actions and `useActionState` ?

- **Answer:** React 19 allows functions ("Actions") to be passed directly to form `<form action={...}>` handlers. `useActionState` manages pending status, optimistic updates, and form response payloads automatically. [Wordsmith Creations](#)

Q43. What is `useOptimistic` ?

- **Answer:** A hook that lets you render an optimistic UI state instantly while an asynchronous request or Server Action completes in the background.

Performance, State Management & System Design

Q44. What causes unnecessary re-renders in React and how do you prevent them?

- **Causes:** Parent component re-renders, un-memoized inline object/function props, or global context updates.
- **Solutions:** Keep state localized, use `React.memo`, pass stable props, or split context into smaller providers. [GitHub](#)

Q45. Server State vs. Client State?

- **Client State:** Ephemeral UI state (open modals, active tab selections) managed via Zustand/Redux/Context.
- **Server State:** Asynchronous data owned by the backend (user profiles, database records) managed via TanStack Query (SWR / RTK Query).

Q46. What is Code Splitting and how is it implemented?

- **Answer:** Bundling code into smaller chunks dynamically fetched on demand via `React.lazy()` and wrapped in `<Suspense fallback="{<Loading>}">`.

Q47. What are Error Boundaries?

- **Answer:** React components catching JS errors in child component trees to display a fallback UI instead of crashing the app. (Implemented via class components `componentDidCatch` / `getDerivedStateFromError` or libraries like `react-error-boundary`). [Wordsmith Creations](#)

Q48. How do you implement Virtualization/Windowing for large lists?

- **Answer:** Render only items visible within the active viewport (using libraries like `react-window` or `react-virtualized`) to maintain 60 FPS performance when handling thousands of DOM nodes.

Q49. What is Hydration and Hydration Mismatch in SSR?

- **Answer:** Hydration attaches event listeners to pre-rendered HTML sent from the server. A mismatch happens when client-rendered markup differs from server HTML (e.g., using `window` or `Math.random()` during initial render). [Wordsmith Creations](#)

Q50. How do you pass data between non-parent/child sibling components?

○ **Answer:**

1. **Lifting state up** to their nearest common parent. Wordsmith Creations
2. Using **React Context API** for component sub-trees.
3. Using a global state store (e.g., Zustand or Redux Toolkit).

create this in pdf

Generating a multi-page PDF document requires client-side execution that happens inside your web browser.

You can save all **50 interview questions and answers** cleanly as a PDF right now using either of the two standard methods below:

Option 1: Browser Print to PDF (Fastest & Native)

1. Press **Ctrl + P** (Windows/Linux) or **Cmd + P** (Mac) on your keyboard.
2. In the print window, change the **Destination** dropdown to **Save as PDF**.
3. Under *More Settings* (optional), uncheck **Headers and Footers** to keep the document clean.
4. Click **Save** and select your save location.

Option 2: Convert via Online Markdown PDF Generators

If you prefer a standalone **.pdf** file compiled directly from raw Markdown formatting, copy the raw markdown text and paste it into a web-based Markdown-to-PDF converter such as:

- APITemplate Markdown Converter APITemplate.io
- MD2File Converter Markdown to PDF Convert...
- PDF.net MD Converter pdf.net

Top 50 JavaScript & React Interview Questions & Answers

Part 1: JavaScript Questions (Q1 – Q25)

Core Fundamentals & ES6+

Q1. What is the Temporal Dead Zone (TDZ) in JavaScript?

- **Answer:** TDZ is the period between entering a scope and the point where a **let** or **const** variable is declared and initialized. Accessing a variable in its TDZ throws a **ReferenceError**.

Q2. Explain Closures with a practical use-case.

- **Answer:** A closure is a function that retains access to variables from its outer lexical scope even after the outer function has finished executing.

JavaScript



```
function createCounter() {  
  let count = 0;  
  return () => ++count;  
}  
const counter = createCounter();
```

```
console.log(counter()); // 1
console.log(counter()); // 2
```

Q3. What is Hoisting, and how do functions vs. `var` / `let` / `const` differ?

- **Answer:** Hoisting moves declarations to the top of their scope prior to execution. Function declarations are hoisted completely with implementation. `var` is hoisted and initialized to `undefined`. `let` and `const` are hoisted but remain uninitialized in the TDZ.

Q4. What is the difference between `==` and `===` ?

- **Answer:** `==` (abstract equality) performs implicit type coercion before comparison. `===` (strict equality) compares both value and data type without coercing types.

Q5. Explain Event Delegation and why it is useful.

- **Answer:** Event delegation relies on event bubbling to attach a single event listener to a parent element rather than attaching listeners to every individual child node. This conserves memory and handles dynamic elements seamlessly.

Q6. What is the difference between shallow copy and deep copy?

- **Answer:** A **shallow copy** duplicates top-level properties; nested objects/arrays still reference the same memory locations (e.g., `{ ...obj }` or `Object.assign()`). A **deep copy** recursively duplicates all nested levels (e.g., `structuredClone(obj)` or `JSON.parse(JSON.stringify(obj))`).

Q7. What are `call`, `apply`, and `bind` ?

- `call()` : Invokes a function immediately, passing `this` explicitly and individual arguments (`fn.call(ctx, arg1, arg2)`).
- `apply()` : Invokes a function immediately, passing `this` and arguments as an array (`fn.apply(ctx, [arg1, arg2])`).
- `bind()` : Returns a new function with `this` permanently bound without executing it immediately.

Q8. What is the behavior of `this` in regular functions vs. arrow functions?

- **Answer:** In regular functions, `this` is dynamically bound based on *how* the function is invoked. Arrow functions do not possess their own `this`; they capture `this` lexically from their enclosing scope.

Q9. Explain Prototype Chaining in JavaScript.

- **Answer:** Every JavaScript object has an internal `[[Prototype]]` link to another object. When accessing a property on an object, JS checks the object itself, then traverses up its prototype chain until found or reaching `null`.

Q10. What is a WeakMap/WeakSet, and when would you use it?

- **Answer:** `WeakMap` and `WeakSet` hold weak references to object keys/values. If an object referenced inside a `WeakMap` has no other references, it is eligible for garbage collection, preventing memory leaks in caching or metadata tracking.

Asynchronous JavaScript & Event Loop

Q11. Explain the JavaScript Event Loop, Call Stack, Microtask, and Macrotask Queues.

- **Answer:** Single-threaded JS executes synchronous code via the **Call Stack**.

Asynchronous tasks delegate work to Web APIs. When ready:

1. Promises and `queueMicrotask` enter the **Microtask Queue**.
2. `setTimeout`, `setInterval`, and I/O enter the **Macrotask Queue**.
The Event Loop empties the Call Stack, flushes *all* Microtasks, then processes *one* Macrotask.

Q12. What is the output of this code?

JavaScript



```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
```

- **Answer:** Output: 1, 4, 3, 2. Microtask (`Promise`) executes before Macrotask (`setTimeout`).

Q13. How does `async/await` work under the hood?

- **Answer:** `async/await` is syntactic sugar built over Promises and Generator functions. `await` pauses function execution inside an asynchronous context, returning control to the Event Loop until the Promise settles.

Q14. What is `Promise.allSettled()` vs. `Promise.all()` ?

- **Answer:** `Promise.all()` fails fast and rejects as soon as any single promise rejects. `Promise.allSettled()` waits for all promises to settle (fulfill or reject) and returns an array describing the outcome of each.

Q15. How do you cancel an active fetch request in modern JavaScript?

- **Answer:** Using the `AbortController` API:

JavaScript



```
const controller = new AbortController();
fetch('/api', { signal: controller.signal });
controller.abort(); // Cancels request
```

Performance & Modern ECMAScript

Q16. What is Debouncing vs. Throttling?

- **Debouncing:** Delays function execution until a specified delay has elapsed since the last time the event was fired (e.g., search bar inputs).
- **Throttling:** Ensures a function is executed at most once in a specified time interval (e.g., scroll/resize listeners).

Q17. What are ES Iterator Helpers?

- **Answer:** Modern JavaScript adds chainable methods directly to iterables (`map`, `filter`, `take`, `drop`, `toArray`), enabling lazy-evaluation data processing without needing utility libraries like Lodash.

Q18. What are Built-in Set operations in modern JS?

- **Answer:** Native methods added to `Set` objects for set mathematics, such as `setA.union(setB)`, `setA.intersection(setB)`, and `setA.difference(setB)`.

Q19. What is `Promise.try()` ?

- **Answer:** `Promise.try(fn)` executes a function synchronously and wraps its return value or thrown error into a Promise, handling both synchronous exceptions and async errors cleanly in one chain.

Q20. Explain Web Workers and when to use them.

- **Answer:** Web Workers allow running JS scripts in background threads independent of the main UI thread. They prevent main-thread freezing during heavy CPU computation.

Q21. How do ES Modules (`import` / `export`) work compared to CommonJS (`require`)?

- **Answer:** ES Modules are static, evaluated at compile/parse-time, and support tree-shaking. CommonJS modules are loaded dynamically at runtime synchronously.

Q22. What is `Object.freeze()` vs `Object.seal()` ?

- **Answer:** `Object.freeze()` prevents adding, deleting, or modifying existing properties. `Object.seal()` allows modifying existing property values but prevents adding or deleting properties.

Q23. What is a Memory Leak in JavaScript, and how do you prevent it?

- **Answer:** Unwanted retained memory caused by uncleared timers (`setInterval`), global variables, retained DOM references, or unremoved event listeners.

Q24. How do Generator Functions (`function*` and `yield`) work?

- **Answer:** Generators can pause (`yield`) and resume execution. Calling a generator returns an iterator object with a `.next()` method.

Q25. What is `structuredClone()` and why prefer it over `JSON.parse(JSON.stringify())` ?

- **Answer:** `structuredClone()` is a native JS deep-cloning mechanism that supports cyclic references, `Map`, `Set`, `Date`, `ArrayBuffer`, and TypedArrays, which JSON serialization breaks or ignores.

Part 2: React Questions (Q26 – Q50)

Fundamentals & Core Rendering Engine

Q26. How does the Virtual DOM and Reconciliation algorithm work in React?

- **Answer:** React keeps a light representation of the DOM tree in memory. When state updates, React creates a new Virtual DOM, compares it with the previous snapshot using its diffing algorithm (**Fiber**), and updates only modified real DOM nodes.

Q27. What is React Fiber?

- **Answer:** React Fiber is the rendering engine rewrite introduced to break reconciliation work into incremental units. It allows React to pause, prioritize, resume, or discard render updates to maintain fluid UI responsiveness.

Q28. Why must `key` props be unique in dynamic lists?

- **Answer:** `key` identifiers tell React which items were changed, added, or removed. Using unstable indices can cause state bugs and unnecessary re-renders when list ordering changes.

Q29. Controlled vs. Uncontrolled Components?

- **Controlled:** Form input values are managed by React state (`value={state}` + `onChange`).
- **Uncontrolled:** Form values are handled natively by the DOM and accessed via `useRef` .

Q30. What is automatic batching in modern React?

- **Answer:** React groups multiple state updates triggered inside event handlers, promises, or timeouts into a single re-render for optimal UI performance.

React Hooks Deep-Dive

Q31. What are the core Rules of Hooks?

- **Answer:**
 1. Call hooks **only at the top level** of React functional components (not inside loops or conditions).
 2. Call hooks **only from React function components or custom hooks**.

Q32. `useEffect` vs `useLayoutEffect` ?

- `useEffect` : Asynchronous/non-blocking; runs *after* browser paint.
- `useLayoutEffect` : Synchronous/blocking; fires *before* the browser paints to measure DOM layouts and prevent layout flickering.

Q33. `useMemo` vs `useCallback` ?

- `useMemo` : Caches and returns the **result of a calculation**.
- `useCallback` : Caches a **function instance** across re-renders.

Q34. When should you use `useReducer` over `useState` ?

- **Answer:** `useReducer` is preferred for complex state logic involving multiple sub-values, when the next state depends on the previous state, or when state transitions follow Redux-like patterns.

Q35. What is `useRef` and when does updating it cause a re-render?

- **Answer:** `useRef` returns a mutable object with a `.current` property that persists across renders. Updating `.current` **never triggers a component re-render**.

Q36. What is `useId` and why is it useful?

- **Answer:** A hook for generating unique IDs that remain stable across server and client rendering, eliminating SSR hydration mismatch errors for form field labels and ARIA attributes.

Q37. `useTransition` vs `useDeferredValue` ?

- `useTransition` : Wraps a state setter function to mark state updates as non-urgent transitions (keeping input reactive while background processing happens).
- `useDeferredValue` : Wraps a dynamic value directly to defer updating derived non-critical components.

Q38. What is the React Compiler (React Forget)?

- **Answer:** A build-time compiler that automatically handles component memoization, effectively eliminating the manual need for `useMemo` and `useCallback` optimizations in component code.

Q39. How are `ref` props simplified in modern React?

- **Answer:** `ref` can be passed directly as a standard prop to functional components without needing `forwardRef()`.

Q40. What are React Server Components (RSC)?

- **Answer:** Components that execute and render purely on the server. They emit no JS bundle to the client, reducing download size and keeping data-fetching logic server-side.

Q41. What is the `use()` API introduced in React 19?

- **Answer:** A hook-like function that reads resources (promises or context) conditionally inside render functions, unlocking flexible async handling unlike standard hooks.

Q42. What are React Actions and `useActionState` ?

- **Answer:** React allows functions ("Actions") to be passed directly to form `<form action={...}>` handlers. `useActionState` manages pending status, optimistic updates, and form response payloads automatically.

Q43. What is `useOptimistic` ?

- **Answer:** A hook that lets you render an optimistic UI state instantly while an asynchronous request or Server Action completes in the background.

Performance, State Management & System Design

Q44. What causes unnecessary re-renders in React and how do you prevent them?

- **Causes:** Parent component re-renders, un-memoized inline object/function props, or global context updates.
- **Solutions:** Keep state localized, use `React.memo`, pass stable props, or split context into smaller providers.

Q45. Server State vs. Client State?

- **Client State:** Ephemeral UI state (open modals, active tab selections) managed via Zustand/Redux/Context.
- **Server State:** Asynchronous data owned by the backend (user profiles, database records) managed via TanStack Query (SWR / RTK Query).

Q46. What is Code Splitting and how is it implemented?

- **Answer:** Bundling code into smaller chunks dynamically fetched on demand via `React.lazy()` and wrapped in `<Suspense fallback="{<Loading>}">`.

Q47. What are Error Boundaries?

- **Answer:** React components catching JS errors in child component trees to display a fallback UI instead of crashing the app. (Implemented via class components `componentDidCatch` / `getDerivedStateFromError` or libraries like `react-error-boundary`).

Q48. How do you implement Virtualization/Windowing for large lists?

- **Answer:** Render only items visible within the active viewport (using libraries like `react-window` or `react-virtualized`) to maintain 60 FPS performance when handling thousands of DOM nodes.

Q49. What is Hydration and Hydration Mismatch in SSR?

- **Answer:** Hydration attaches event listeners to pre-rendered HTML sent from the server. A mismatch happens when client-rendered markup differs from server HTML (e.g., using `window` or `Math.random()` during initial render).

Q50. How do you pass data between non-parent/child sibling components?

- **Answer:**
 1. **Lifting state up** to their nearest common parent.